

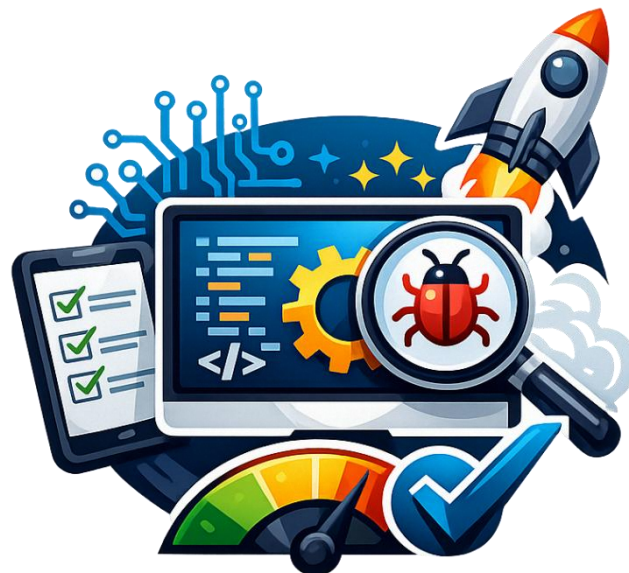
Curso Técnico em
Desenvolvimento de Sistemas

UC | Teste de Sistemas

APOSTILA COMPLETA

Fundamentos de Teste de Software + Python para Testes

2025 / 2026



MODULO 1
Fundamentos
Aulas 1 e 2 | 10h

MODULO 2
Testes Manuais
Aulas 3 e 4 | 10h

MODULO 3
Python + Automacao
Aulas 5 a 11 | 40h

Sumario

MODULO 1 - FUNDAMENTOS DE TESTE DE SOFTWARE

Aula 1 - O Que e Teste de Software?

Aula 2 - Tipos, Niveis e Normas de Teste

MODULO 2 - TESTES MANUAIS E DOCUMENTACAO

Aula 3 - Plano de Teste e Casos de Teste

Aula 4 - Identificacao de Bugs e Relatorio de Defeito

MODULO 3 - PYTHON E AUTOMACAO DE TESTES

Aula 5 - Introducao ao Python

Aula 6 - Funcoes, Modulos e Tratamento de Erros

Aula 7 - Testes Unitarios com pytest

Aula 8 - Testes de Interface Web com Playwright

Aula 9 - Relatorios de Teste Automatizados

Aula 10 - Integracao Continua e Boas Praticas

Aula 11 - Projeto Integrador Final

Apresentacao da Apostila

Bem-vindo(a) a Unidade Curricular de Teste de Sistemas! Esta apostila foi elaborada especialmente para voce, aluno(a) do curso Tecnico em Desenvolvimento de Sistemas do SENAI.

Voce ja passou por diversas UCs importantes: Banco de Dados, Logica de Programacao, IoT, Programacao de Aplicativos e Codificacao para Front End. Agora chegou a hora de aprender a garantir que tudo o que voce desenvolve funciona corretamente.

O que voce vai aprender nesta UC?

- Como planejar e documentar roteiros de teste profissionais
- Os principais tipos e niveis de teste utilizados na industria
- Normas tecnicas de qualidade de software (ISO/IEC 29119)
- Como escrever casos de teste, identificar bugs e documentar resultados
- Python do zero: a linguagem mais usada em automacao de testes
- Testes unitarios com pytest
- Testes automatizados de interface web com Playwright

Geracao de relatorios e boas praticas de CI/CD

Como esta organizada esta apostila?

Modulo	Conteudo
Modulo 1 (Aulas 1-2)	Fundamentos: o que e teste, por que testamos, tipos e normas
Modulo 2 (Aulas 3-4)	Testes manuais: plano de teste, casos de teste, bug reports
Modulo 3 (Aulas 5-11)	Python + automacao: pytest, Playwright, relatorios, CI



Importante

Esta apostila usa a linguagem Python para automacao de testes, pois ela sera fundamental nas proximas UCs do curso (Modelagem, Desenvolvimento e Implantacao de Sistemas). Todo o conteudo foi pensado para Windows 11 com VS Code, as ferramentas que voce ja usa.

MODULO 1

Fundamentos de Teste de Software

Aulas 1 e 2 | 10 horas

AULA 1

O Que e Teste de Software?

1.1 Introducao: Por que testamos software?

Imagine que voce construiu um site de e-commerce para uma loja. O cliente clica em 'Comprar' e nada acontece. Ou pior: o valor do produto aparece errado e a loja perde dinheiro em cada venda. Situacoes assim sao chamadas de falhas de software, e elas acontecem todos os dias no mundo real.

O teste de software e o conjunto de atividades que nos permite encontrar esses problemas ANTES que o usuario final seja afetado. E como uma revisao antes de entregar um trabalho escolar — so que para sistemas de computador.

#

Definicao

Teste de Software: processo de verificacao e validacao de um sistema ou componente para determinar se ele satisfaz os requisitos especificados e identificar diferencas entre comportamento esperado e obtido. (IEEE 829)

1.2 O Custo do Bug: Quanto custa nao testar?

Estudos da industria mostram que o custo de corrigir um defeito aumenta exponencialmente quanto mais tarde ele e encontrado:

Quando o bug e encontrado	Custo relativo de correcao
Durante os requisitos	1x (mais barato)
Durante o design	3x
Durante a codificacao	5x
Durante o teste	10x
Apos o lancamento (producao)	100x (mais caro)

!

Importante

A NASA calculou que o bug no codigo do foguete Ariane 5 (1996) custou US\$ 370 milhoes. Em 2012, o banco Knight Capital perdeu US\$ 440 milhoes em 45 minutos por um bug nao testado.

Testar cedo e testar frequentemente e sempre mais barato do que corrigir em producao.

1.3 O Que é um Bug, Defeito, Erro e Falha?

Esses termos são usados no dia a dia e têm significados técnicos precisos:

Termo	Definição e Exemplo
Erro (Error)	Engano humano durante a criação do código. Ex: programador digitou '+' em vez de '-'.
Defeito (Defect/Bug)	Resultado do erro no código-fonte. Ex: a função de desconto está subtraindo errado.
Falha (Failure)	Comportamento incorreto observado na execução. Ex: o preço no carrinho está maior que o esperado.
Incidente	Qualquer situação que requer investigação, podendo ou não ser uma falha.



Dica

Lembre-se: o programador comete um ERRO, esse erro gera um DEFEITO no código, e quando o defeito é executado o sistema exibe uma FALHA.

1.4 Princípios Fundamentais de Teste (ISTQB)

O ISTQB (International Software Testing Qualifications Board) define 7 princípios que todo testador deve conhecer:

1. Teste demonstra a presença de defeitos, não a sua ausência — testar pode mostrar que existem bugs, mas não pode provar que o sistema está 100% correto.
2. Teste exaustivo é impossível — não dá para testar todas as combinações possíveis; é preciso priorizar riscos e importância.
3. Teste antecipado economiza tempo e dinheiro — comece a pensar em testes desde os requisitos.
4. Agrupamento de defeitos — a maioria dos bugs se concentra em poucos módulos (regra 80/20).
5. Paradoxo do pesticida — se você sempre executa os mesmos testes, os bugs sobreviventes serão imunes a eles; revise e crie novos testes.
6. Teste depende do contexto — testar um app bancário é muito diferente de testar um jogo mobile.
7. Ausência de erros é uma ilusão — um sistema sem bugs ainda pode ser inutilizável se não atender às necessidades do usuário.

1.5 Ciclo de Vida do Desenvolvimento de Software (SDLC) e Testes

Os testes se encaixam em todas as fases do desenvolvimento:

Fase do SDLC	Atividade de Teste Correspondente
Levantamento de Requisitos	Revisão de requisitos, identificação de ambiguidades
Design / Modelagem	Revisão de arquitetura, criação do plano de teste
Codificação	Testes unitários, revisão de código
Integração	Testes de integração

Homologacao	Testes de sistema, testes de aceitacao
Implantacao	Testes de regressao, smoke tests
Manutencao	Testes de regressao apos correcoes

1.6 Atividade Pratica - Aula 1

*

Atividade 1.1 - Identificando Bugs no Mundo Real

Pesquise na internet um caso famoso de falha de software (pode ser o Ariane 5, o erro do sistema de saude do Reino Unido, o bug do Y2K, ou qualquer outro).

Responda em um documento Word:

- Qual era o sistema envolvido?
- Qual foi o erro humano cometido?
- Qual defeito esse erro gerou no codigo?
- Qual foi a falha observada pelo usuario?
- Qual foi o impacto financeiro ou humano?
- Como um teste poderia ter evitado o problema?

AULA 2

Tipos, Níveis e Normas de Teste

2.1 Níveis de Teste

Os testes são organizados em níveis que correspondem a diferentes partes do sistema sendo testado:

Nível	O que testa / Exemplo
Teste Unitário	Testa a menor unidade de código isolada (uma função, um método). Ex: testar se a função <code>calcular_desconto()</code> retorna o valor correto.
Teste de Integração	Testa a comunicação entre dois ou mais componentes. Ex: testar se o módulo de pagamento se comunica corretamente com o banco de dados.
Teste de Sistema	Testa o sistema completo como um todo, do ponto de vista do usuário. Ex: fazer um cadastro, login, compra e logout completos.
Teste de Aceitação (UAT)	Testa se o sistema atende as necessidades do cliente. Geralmente feito pelo próprio cliente antes de aceitar a entrega.

2.2 Tipos de Teste

Além dos níveis, os testes também se diferenciam pelo objetivo:

Tipo	O que verifica
Teste Funcional	Se as funcionalidades fazem o que o requisito especifica. Ex: clicar em 'Login' com dados corretos deve abrir o dashboard.
Teste Não-Funcional	Performance, segurança, usabilidade, compatibilidade. Ex: o sistema deve suportar 1000 usuários simultâneos.
Teste de Regressão	Se uma nova alteração no código quebrou algo que funcionava antes. Fundamental após correções de bugs.
Smoke Test	Verificação rápida das funcionalidades mais críticas após um novo deploy. Ex: o site abre? O login funciona?
Teste Exploratório	O testador explora o sistema sem roteiro fixo, usando criatividade para encontrar bugs inesperados.
Teste de Carga/Performance	Comportamento do sistema sob condições extremas de uso. Ferramentas: JMeter, k6.
Teste de Segurança	Vulnerabilidades e brechas. Ex: injeção de SQL, XSS, autenticação fraca.
Teste de Usabilidade	Se o sistema é fácil e intuitivo de usar. Geralmente feito com usuários reais.

2.3 Técnicas de Teste

As técnicas definem como projetamos os casos de teste:

Caixa Preta (Black Box)

O testador não conhece o código interno. Testa apenas as entradas e saídas do sistema. É a perspectiva do usuário.

- Particionamento de equivalência: dividir entradas em grupos válidos e inválidos
- Análise de valor limite: testar nos limites dos intervalos (0, 1, 99, 100)
- Tabela de decisão: combinações de condições e ações

Caixa Branca (White Box)

O testador conhece o código e cria testes para cobrir caminhos específicos do código-fonte.

- Cobertura de linhas: garantir que cada linha do código seja executada
- Cobertura de ramos: garantir que cada condição IF seja testada como verdadeira e falsa

Caixa Cinza (Grey Box)

Combinação das duas abordagens: o testador tem conhecimento parcial do sistema (ex: estrutura do banco de dados, mas não o código).

2.4 Norma ISO/IEC 29119

A norma ISO/IEC 29119 é o padrão internacional para teste de software. Ela define vocabulário, processos, documentação e técnicas de teste.

Parte da norma	O que abrange
ISO 29119-1	Conceitos e vocabulário de teste
ISO 29119-2	Processos de teste (planejamento, execução, relatórios)
ISO 29119-3	Documentação de teste (plano, caso de teste, relatório)
ISO 29119-4	Técnicas de teste
ISO 29119-5	Teste baseado em palavras-chave (automação)

Definição

A norma ISO/IEC 29119 não é uma receita rígida, mas um guia de boas práticas. Muitas empresas adaptam seus processos baseando-se nela.

2.5 Ambiente de Teste

Para testar software, precisamos de um ambiente adequado. Isso inclui:

- Hardware e sistema operacional compatível com o sistema em teste
- Software de suporte (banco de dados, servidores, navegadores)
- Dados de teste realistas, mas anonimizados (nunca usar dados reais de clientes em testes)
- Ferramentas de gestão de bugs (GitHub Issues, Jira, Trello)
- Ferramentas de automação (pytest, Playwright, Selenium)
- Controle de versão (Git/GitHub para manter scripts de teste)

2.6 Atividade Prática - Aula 2

* Atividade 2.1 - Mapa Mental de Tipos de Teste

Usando o Canva, Miro, ou papel mesmo, crie um mapa mental com os tipos e níveis de teste.

O mapa deve conter no mínimo:

- Os 4 níveis de teste (unitário, integração, sistema, aceitação)
- 5 tipos de teste com exemplos próprios
- As 3 técnicas (caixa preta, branca, cinza)

* Atividade 2.2 - Classificando Testes

Para cada situação abaixo, identifique o nível e o tipo de teste mais adequado:

- Verificar se a função soma(2,3) retorna 5
- Verificar se o sistema aguenta 500 usuários logados ao mesmo tempo
- Fazer um pedido completo no e-commerce (do carrinho ao pagamento)
- Verificar se ao corrigir o bug do login, o cadastro ainda funciona
- O cliente final testa o sistema antes de aprovar a entrega

MODULO 2

Testes Manuais e Documentacao

Aulas 3 e 4 | 10 horas

AULA 3

Plano de Teste e Casos de Teste

3.1 O Plano de Teste

O Plano de Teste é o documento-mãe do processo de teste. Ele define o QUE vai ser testado, COMO, QUANDO, POR QUEM e com QUAIS RECURSOS. É o primeiro documento que um profissional de QA produz ao iniciar um projeto.

#

Definicao

Plano de Teste (Test Plan): documento que descreve o escopo, a abordagem, os recursos e o cronograma das atividades de teste. Baseado na norma ISO/IEC 29119-3.

3.2 Estrutura do Plano de Teste (ISO 29119-3)

Secao	O que deve conter
1. Identificacao	Nome do projeto, versao, data, responsaveis
2. Introducao	Objetivo do plano, sistemas envolvidos, referencias
3. Escopo	O que SERA testado e o que NAO sera testado (out of scope)
4. Abordagem	Niveis de teste, tipos de teste, tecnicas utilizadas
5. Criterios de Entrada	Condicoes para iniciar os testes (ex: build aprovado, ambiente pronto)
6. Criterios de Saida	Condicoes para encerrar os testes (ex: 95% dos casos passaram, bugs criticos = 0)
7. Recursos	Equipe, ferramentas, hardware, dados de teste
8. Cronograma	Datas e duracao de cada fase de teste
9. Riscos	Riscos identificados e plano de mitigacao
10. Aprovacao	Assinaturas dos responsaveis

3.3 Exemplo de Plano de Teste Simplificado

Vamos criar um Plano de Teste para um sistema de login que voce desenvolveu na UC de Front End:

#

Plano de Teste - Sistema de Login (Exemplo)

Projeto: Sistema de Login - Versao 1.0

Data: 01/08/2025 | Responsavel: [Seu Nome]

ESCOPO:

Sera testado: pagina de login, validacao de campos, mensagens de erro, redirecionamento
Nao sera testado: desempenho, seguranca avancada, compatibilidade mobile

TIPOS DE TESTE: Funcional (caixa preta), Regressao

NIVEL: Sistema

CRITERIO DE ENTRADA: Pagina de login deployada no servidor local, banco de dados populado com usuarios de teste

CRITERIO DE SAIDA: 100% dos casos de teste executados, 0 bugs criticos abertos

AMBIENTE: Windows 11, Chrome 125+, VS Code, Node.js 20

3.4 Casos de Teste (Test Cases)

Um caso de teste e uma especificacao precisa de como executar um teste especifico. E o documento mais operacional do processo de teste — e o que o testador usa na hora de executar os testes.

Definicao

Caso de Teste: conjunto de condicoes de entrada, acoes e resultados esperados desenvolvido para verificar um aspecto especifico do software. Cada caso de teste deve ser independente e reproduzivel.

3.5 Estrutura de um Caso de Teste

Campo	Descricao / Exemplo
ID	Identificador unico. Ex: CT-001
Titulo	Descricao curta do que e testado. Ex: Login com credenciais validas
Pre-condicoes	O que precisa estar pronto antes de executar. Ex: usuario 'admin@teste.com' cadastrado
Dados de entrada	Valores exatos a inserir. Ex: Email: admin@teste.com Senha: Senha@123
Passos	Acoes numeradas e precisas que o testador deve executar
Resultado Esperado	O que DEVE acontecer se o sistema estiver correto
Resultado Obtido	O que REALMENTE aconteceu (preenchido na execucao)
Status	PASSOU / FALHOU / BLOQUEADO / NAO EXECUTADO
Observacoes	Evidencias, prints, logs, comentarios

3.6 Exemplo Completo de Casos de Teste

Vamos criar casos de teste para a pagina de login do nosso projeto:

CT-001 | Login com credenciais validas

Pre-condicoes: Usuario admin@teste.com com senha Senha@123 cadastrado no banco de dados

Dados de entrada: Email: admin@teste.com | Senha: Senha@123

Passos:

1. Abrir o navegador e acessar http://localhost:3000/login
2. Digitar 'admin@teste.com' no campo Email
3. Digitar 'Senha@123' no campo Senha
4. Clicar no botao 'Entrar'

Resultado Esperado: Usuario e redirecionado para o dashboard em /dashboard, nome do usuario exibido no cabecalho

Status: [PASSOU/FALHOU]

CT-002 | Login com senha incorreta

Pre-condicoes: Mesmas do CT-001

Dados de entrada: Email: admin@teste.com | Senha: senhaerrada

Passos:

1. Acessar http://localhost:3000/login
2. Digitar 'admin@teste.com' no campo Email
3. Digitar 'senhaerrada' no campo Senha
4. Clicar no botao 'Entrar'

Resultado Esperado: Mensagem de erro 'Email ou senha incorretos' exibida, usuario permanece na pagina de login

Status: [PASSOU/FALHOU]

CT-003 | Login com campo de email vazio

Dados de entrada: Email: (vazio) | Senha: Senha@123

Passos: 1. Acessar /login | 2. Deixar Email em branco | 3. Digitar senha | 4. Clicar Entrar

Resultado Esperado: Mensagem de validacao 'Email e obrigatorio' exibida, foco retorna ao campo Email

Status: [PASSOU/FALHOU]

3.7 Atividade Pratica - Aula 3

* Atividade 3.1 - Criando seu Plano e Casos de Teste

Escolha uma das paginas que voce desenvolveu na UC de Front End (login, cadastro, carrinho, etc.)

Tarefa 1: Elabore um Plano de Teste simplificado para essa pagina (use o modelo da secao 3.2)

Tarefa 2: Crie no minimo 6 casos de teste para essa pagina, cobrindo:

- 2 cenarios de sucesso (caminho feliz)
- 2 cenarios de erro (dado invalido)
- 2 cenarios de limite (campo vazio, dado muito longo, etc.)

Tarefa 3: Execute os casos de teste manualmente e preencha os campos 'Resultado Obtido' e 'Status'

Entrega: Documento Word com Plano de Teste + planilha/tabela com os casos de teste

AULA 4

Identificacao de Bugs e Relatorio de Defeito

4.1 Ciclo de Vida de um Defeito

Quando um testador encontra um bug, ele não pode simplesmente falar 'achei um problema'. Existe um processo formal para registrar, rastrear e resolver defeitos:

Status do Defeito	Significado
Novo (New)	Bug foi encontrado e registrado pelo testador
Aberto (Open)	Bug foi confirmado e atribuído a um desenvolvedor
Em Andamento (In Progress)	Desenvolvedor está corrigindo o bug
Corrigido (Fixed)	Desenvolvedor terminou a correção e entregou para re-teste
Re-teste (Retest)	Testador verifica se a correção funcionou
Fechado (Closed)	Bug foi corrigido e verificado com sucesso
Reaberto (Reopened)	A correção não funcionou — bug retorna para o desenvolvedor
Duplicado (Duplicate)	Já existe outro relatório para o mesmo bug
Não é Bug (Not a Bug)	O comportamento era o esperado (mal-entendido de requisito)

4.2 Classificacao de Severidade e Prioridade

Não são todos os bugs iguais. Precisamos classificar para saber o que corrigir primeiro:

Severidade	Descrição / Exemplo
Critico (Critical)	O sistema falha completamente ou perde dados. Ex: pagamento processa sem confirmar pedido.
Alto (High)	Funcionalidade importante quebrada sem solução alternativa. Ex: não é possível fazer login.
Medio (Medium)	Funcionalidade com problemas, mas há uma solução alternativa. Ex: botão de filtro não funciona, mas dá para buscar manualmente.
Baixo (Low)	Problema cosmético ou de baixo impacto. Ex: texto com erro de ortografia, botão levemente desalinhado.



Dica

SEVERIDADE x PRIORIDADE: severidade é o impacto técnico do bug. Prioridade é a urgência de correção (pode ser influenciada por fatores de negócio). Um bug de texto errado no logo da empresa pode ter BAIXA severidade mas ALTA prioridade.

4.3 Relatorio de Defeito (Bug Report)

Um bom relatório de defeito é tão importante quanto encontrar o bug. Um relatório ruim desperdiça tempo do desenvolvedor. Um bom relatório permite reprodução e correção rápida.

Definição

Regra de Ouro do Bug Report: o desenvolvedor deve conseguir reproduzir o bug SEM precisar perguntar mais nada ao testador. Se precisar perguntar, o relatório é incompleto.

4.4 Estrutura do Bug Report

Campo	Como preencher
ID	Número único gerado automaticamente. Ex: BUG-042
Título	Descrição curta e objetiva. Ex: 'Botão Comprar não responde no Firefox 125'
Descrição	Explicação detalhada do problema
Passos para Reproduzir	Passo a passo numerado e preciso para reproduzir o bug
Resultado Esperado	O que deveria acontecer
Resultado Obtido	O que realmente aconteceu
Severidade	Crítico / Alto / Médio / Baixo
Prioridade	Alta / Média / Baixa
Ambiente	OS, navegador, versão do sistema. Ex: Windows 11, Chrome 125.0.6422.76
Evidências	Print de tela, vídeo, logs de console, URL da página
Reportado por	Nome do testador e data
Atribuído a	Nome do desenvolvedor responsável

4.5 Exemplo de Bug Report Completo

BUG-042 | Botão Comprar não processa pagamento no Firefox

Descrição: Ao clicar no botão 'Finalizar Compra' na página de checkout, nenhuma ação é executada. O console do navegador exibe o erro 'TypeError: Cannot read properties of undefined (reading click)'.

Passos para Reproduzir:

1. Acessar <http://localhost:3000/shop>
2. Adicionar qualquer produto ao carrinho
3. Clicar em 'Ver Carrinho'
4. Preencher todos os dados de entrega
5. Clicar em 'Finalizar Compra'

Resultado Esperado: Modal de confirmação de pagamento e exibido

Resultado Obtido: Nada acontece. Console exibe TypeError.

Severidade: CRÍTICO | Prioridade: ALTA

Ambiente: Windows 11, Firefox 126.0, Node.js 20.11, localhost:3000

Evidências: [print_bug042.png] [console_log_bug042.txt]

Reportado por: Aluno Teste | 05/08/2025

4.6 Ferramentas de Gestao de Bugs

No mercado, os bugs sao gerenciados em ferramentas especificas. As mais comuns sao:

Ferramenta	Caracteristicas
GitHub Issues	Gratuito, integrado ao repositorio Git. Ideal para projetos open source e estudantes.
Jira	Ferramenta profissional mais usada no mercado. Paga, mas tem plano gratuito para times pequenos.
Trello	Simples, visual, baseado em Kanban. Bom para times pequenos.
Azure DevOps	Suite completa da Microsoft. Integra com Visual Studio e Azure.
Bugzilla	Open source, muito usado em projetos Mozilla.



Dica

Durante o curso, usaremos GitHub Issues para gestao de bugs. Assim voce ja pratica Git e controle de defeitos ao mesmo tempo — habilidades muito valorizadas no mercado.

4.7 Tutorial: Criando um Bug Report no GitHub Issues

Vamos aprender a registrar um bug usando GitHub Issues, a ferramenta que usaremos nas atividades praticas:

8. Acesse github.com e faca login na sua conta
9. Abra o repositorio do projeto (ou crie um novo para fins de pratica)
10. Clique na aba 'Issues' no menu superior do repositorio
11. Clique no botao verde 'New Issue'
12. No campo 'Title', escreva o titulo do bug. Ex: 'Botao de login nao funciona no Safari'
13. No campo de descricao, use o template de bug report (passos, resultado esperado, resultado obtido, ambiente)
14. No painel direito, clique em 'Labels' e adicione a label 'bug'
15. Se souber quem deve corrigir, clique em 'Assignees' e atribua o desenvolvedor
16. Clique em 'Submit new issue'



Dica

Dica profissional: crie um template de bug report no seu repositorio (.github/ISSUE_TEMPLATE/bug_report.md) para padronizar todos os relatorios da equipe automaticamente.

4.8 Atividade Pratica - Aula 4



Atividade 4.1 - Cacada aos Bugs

Use a pagina web que voce desenvolveu na UC de Front End ou um site que o professor indicar.

Tarefa 1: Realize teste exploratorio por 30 minutos, sem roteiro fixo

Tarefa 2: Registre todos os problemas encontrados (minimo 3 bugs) usando o modelo de Bug Report

Tarefa 3: Crie uma conta no GitHub (se ainda nao tiver) e registre os bugs encontrados como GitHub Issues

Tarefa 4: Classifique cada bug por severidade e prioridade

Entrega: Link do repositório GitHub com os Issues criados + documento com justificativa das classificacoes

MODULO 3

Python e Automacao de Testes

Aulas 5 a 11 | 40 horas

AULA 5

Introducao ao Python

5.1 Por que Python para Testes?

Python é a linguagem de programação mais usada no mundo de QA e automação de testes. Veja por que:

Vantagem	Detalhes
Sintaxe simples e legível	Código Python se lê quase como português. Ideal para quem está começando.
Ecossistema de testes maduro	pytest, unittest, Playwright, Selenium, Robot Framework — todas as ferramentas líderes suportam Python.
Amplamente adotado no mercado	Empresas como Google, Netflix, Spotify e iFood usam Python extensivamente em suas equipes de QA.
Versátil	Serve para testes web, APIs, mobile, banco de dados, e também para as próximas UCs do curso.
Gratuito e open source	Sem custo de licença, vasta documentação e comunidade ativa.

5.2 Instalando o Ambiente Python no Windows 11

Vamos configurar tudo passo a passo. Siga com atenção cada etapa:

Passo 1: Instalar o Python

17. Abra o navegador e acesse python.org/downloads
18. Clique em 'Download Python 3.x.x' (versão mais recente)
19. Execute o instalador baixado
20. MUITO IMPORTANTE: marque a opção 'Add Python to PATH' antes de clicar em Install
21. Clique em 'Install Now' e aguarde a instalação
22. Após concluir, clique em 'Close'

**Atenção**

Se você NÃO marcar 'Add Python to PATH', o terminal não vai reconhecer o comando 'python'. Se isso acontecer, desinstale e reinstale marcando a opção.

Passo 2: Verificar a instalação

23. Abra o Terminal do Windows (pressione Win + R, digite 'cmd', pressione Enter)
24. Digite o comando abaixo e pressione Enter:

```
1 python --version
```

25. Você deve ver algo como: Python 3.12.x

26. Teste também o pip (gerenciador de pacotes):

```
1 pip --version
```

Passo 3: Configurar o VS Code para Python

27. Abra o VS Code

28. Pressione Ctrl+Shift+X para abrir a loja de extensões

29. Busque por 'Python' (extensão da Microsoft)

30. Clique em 'Install'

31. Busque também por 'Pylance' e instale



Dica

O VS Code com a extensão Python oferece autocompletar inteligente, debug integrado e execução de testes direto do editor — muito mais produtivo do que o editor padrão do Python (IDLE).

5.3 Variáveis e Tipos de Dados

Em Python, variáveis são criadas no momento em que recebem um valor. Não é necessário declarar o tipo antecipadamente:

```
1 # Variáveis em Python - sem necessidade de declarar tipo
2 nome = "Maria"      # string (texto)
3 idade = 17          # int (numero inteiro)
4 nota = 8.5           # float (numero decimal)
5 aprovado = True      # bool (verdadeiro/falso)
6
7 # Verificando o tipo de uma variável
8 print(type(nome))    # <class 'str'>
9 print(type(idade))   # <class 'int'>
10 print(type(nota))    # <class 'float'>
11 print(type(aprovado)) # <class 'bool'>
```

```
1 # Strings: texto em Python
2 primeiro_nome = "Carlos"
3 sobrenome = 'Silva'
4 nome_completo = primeiro_nome + " " + sobrenome # concatenacao
5 print(nome_completo) # Carlos Silva
6
7 # F-strings: forma moderna de formatar strings (Python 3.6+)
8 idade = 20
9 mensagem = f"Meu nome e {nome_completo} e tenho {idade} anos."
10 print(mensagem) # Meu nome e Carlos Silva e tenho 20 anos.
11
12 # Metodos uteis de string
13 texto = " Ola Mundo "
14 print(texto.strip()) # 'Ola Mundo' (remove espacos das bordas)
```

```
15 print(texto.upper()) # ' OLA MUNDO '  
16 print(texto.lower()) # ' ola mundo '  
17 print(texto.replace("Mundo", "Python")) # ' Ola Python '
```

5.4 Listas, Tuplas e Dicionarios

```
1 # LISTA: colecao ordenada e modificavel  
2 frutas = ["maca", "banana", "uva"]  
3 frutas.append("laranja") # adiciona ao final  
4 frutas.remove("banana") # remove pelo valor  
5 print(frutas[0]) # 'maca' (indice começa em 0)  
6 print(len(frutas)) # 3 (quantidade de itens)  
7  
8 # Iterando sobre uma lista  
9 for fruta in frutas:  
10     print(f"Fruta: {fruta}")  
11  
12 # DICCIONARIO: pares chave-valor (como objetos em JavaScript)  
13 usuario = {  
14     "nome": "Ana Lima",  
15     "email": "ana@email.com",  
16     "ativo": True,  
17     "nivel": 3  
18 }  
19 print(usuario["nome"]) # Ana Lima  
20 print(usuario.get("email")) # ana@email.com  
21 usuario["nivel"] = 4 # modificar valor  
22 usuario["cidade"] = "Brasilia" # adicionar novo campo
```

5.5 Condicionais

```
1 # IF / ELIF / ELSE  
2 nota = 7.5  
3  
4 if nota >= 9:  
5     print("Excelente!")  
6 elif nota >= 7:  
7     print("Aprovado")  
8 elif nota >= 5:  
9     print("Recuperacao")  
10 else:  
11     print("Reprovado")  
12  
13 # IMPORTANTE: Python usa INDENTACAO (espacos) para definir blocos  
14 # Nao use chaves {} como em JavaScript!  
15  
16 # Operadores de comparacao  
17 # == igual a    != diferente de  
18 # > maior que  < menor que
```

```
19 # >= maior ou igual <= menor ou igual
20
21 # Operadores logicos
22 x = 10
23 if x > 5 and x < 20: # E logico
24     print("x esta entre 5 e 20")
25
26 if x < 0 or x > 100: # OU logico
27     print("x esta fora do intervalo")
28
29 if not (x == 10): # NAO logico
30     print("x nao e 10")
```

5.6 Loops

```
1 # FOR: para iterar sobre uma sequencia
2 for i in range(5): # 0, 1, 2, 3, 4
3     print(f"Iteracao {i}")
4
5 for i in range(1, 6): # 1, 2, 3, 4, 5
6     print(f"Numero {i}")
7
8 # Iterando com indice
9 frutas = ["maca", "banana", "uva"]
10 for indice, fruta in enumerate(frutas):
11     print(f"{indice}: {fruta}")
12
13 # WHILE: enquanto uma condicao for verdadeira
14 tentativas = 0
15 senha_correta = "python123"
16
17 while tentativas < 3:
18     senha = input("Digite a senha: ")
19     if senha == senha_correta:
20         print("Acesso liberado!")
21         break # sai do loop
22     else:
23         tentativas += 1
24     print(f"Senha incorreta. Tentativas restantes: {3 - tentativas}")
```

5.7 Atividade Pratica - Aula 5

*

Atividade 5.1 - Primeiros Scripts Python

Crie uma pasta 'aula05' no VS Code e crie os seguintes scripts:

Script 1 (calculadora.py): Crie um programa que leia dois numeros e uma operacao (+, -, *, /) e exiba o resultado. Use condicionais para tratar divisao por zero.

Script 2 (cadastro.py): Crie um programa que armazene 3 usuarios em uma lista de dicionarios (nome, email, ativo: True/False). Liste apenas os usuarios ativos usando um loop.

Script 3 (notas.py): Leia as notas de 5 alunos, calcule a media, e classifique cada um como Aprovado (≥ 7), Recuperacao (≥ 5) ou Reprovado (< 5).

AULA 6

Funcoes, Modulos e Tratamento de Erros

6.1 Funcoes em Python

Funcoes sao blocos de codigo reutilizaveis. Em Python, definimos funcoes com a palavra-chave 'def':

```
1  # Definindo uma funcao simples
2  def saudar(nome):
3      return f'Ola, {nome}! Bem-vindo(a)!'
4
5  # Chamando a funcao
6  mensagem = saudar("Carlos")
7  print(mensagem) # Ola, Carlos! Bem-vindo(a)!
8
9  # Funcao com multiplos parametros e valor padrao
10 def calcular_desconto(preco, desconto=0.1):
11     """Calcula o preco com desconto.
12     Args:
13         preco: valor original do produto
14         desconto: percentual de desconto (padrao: 10%)
15     Returns:
16         preco apos aplicar o desconto
17     """
18     valor_desconto = preco * desconto
19     preco_final = preco - valor_desconto
20     return preco_final
21
22 print(calcular_desconto(100)) # 90.0 (10% desconto)
23 print(calcular_desconto(100, 0.20)) # 80.0 (20% desconto)
```



Dica

A 'docstring' (texto entre aspas triplas logo abaixo do def) e a forma padrao de documentar funcoes Python. Ela aparece quando voce usa help(funcao) e e fundamental para manutencao do codigo.

6.2 Funcoes Relevantes para Testes

```
1  # Funcoes de validacao (muito usadas em testes)
2  def validar_email(email):
3      """Valida formato basico de email."""
4      return "@" in email and "." in email.split("@")[-1]
5
6  def validar_senha(senha):
7      """Valida se senha tem pelo menos 8 caracteres.
8      Returns dict com resultado e mensagem.
```

```
9      """
10     if len(senha) < 8:
11         return {"valido": False, "mensagem": "Senha muito curta"}
12     if not any(c.isupper() for c in senha):
13         return {"valido": False, "mensagem": "Senha precisa de maiuscula"}
14     if not any(c.isdigit() for c in senha):
15         return {"valido": False, "mensagem": "Senha precisa de numero"}
16     return {"valido": True, "mensagem": "Senha valida"}
17
18 # Testando manualmente
19 print(validar_email("usuario@email.com")) # True
20 print(validar_email("emailinvalido"))    # False
21 print(validar_senha("Abc12345"))         # {'valido': True, ...}
22 print(validar_senha("123"))              # {'valido': False, ...}
```

6.3 Tratamento de Excecoes (try/except)

Erros em Python sao chamados de excecoes. Podemos captura-los com try/except para evitar que o programa quebre:

```
1 # Sem tratamento: o programa quebra com erro feio
2 # resultado = 10 / 0 # ZeroDivisionError: division by zero
3
4 # COM tratamento: controlamos o que acontece
5 def dividir(a, b):
6     try:
7         resultado = a / b
8         return resultado
9     except ZeroDivisionError:
10         return "Erro: divisao por zero nao e permitida"
11     except TypeError:
12         return "Erro: os valores devem ser numericos"
13
14 print(dividir(10, 2)) # 5.0
15 print(dividir(10, 0)) # Erro: divisao por zero...
16 print(dividir(10, "x")) # Erro: os valores devem ser numericos
17
18 # try/except/else/finally
19 def ler_arquivo(caminho):
20     try:
21         arquivo = open(caminho, 'r', encoding='utf-8')
22         conteudo = arquivo.read()
23     except FileNotFoundError:
24         return "Arquivo nao encontrado"
25     else:
26         arquivo.close()
27         return conteudo
28     finally:
29         print("Operacao de leitura concluida") # executa sempre
```

6.4 Modulos e Importacoes

```
1 # Python tem muitos modulos prontos para usar
2 import math
3 import random
4 import datetime
5 import os
6
7 # Usando modulos
8 print(math.sqrt(16))      # 4.0
9 print(math.pi)           # 3.14159...
10 print(random.randint(1, 10)) # numero aleatorio entre 1 e 10
11
12 # Data e hora - muito util em testes
13 agora = datetime.datetime.now()
14 print(agora.strftime("%d/%m/%Y %H:%M:%S")) # 01/08/2025 14:30:00
15
16 # Importando apenas o necessario
17 from math import sqrt, ceil, floor
18 print(sqrt(25))          # 5.0 (sem precisar de math.)
19
20 # Criando seu proprio modulo
21 # Salve como 'validacoes.py' e importe em outros arquivos:
22 # from validacoes import validar_email, validar_senha
```

6.5 Atividade Pratica - Aula 6

* Atividade 6.1 - Modulo de Validacoes

Crie um arquivo chamado 'validacoes.py' com as seguintes funcoes:

1. validar_cpf(cpf): verifica se o CPF tem 11 digitos numericos
2. validar_data(data_str): verifica se a string esta no formato DD/MM/AAAA
3. validar_telefone(tel): verifica se tem 10 ou 11 digitos numericos
4. calcular_imc(peso, altura): calcula o IMC e retorna o valor e a classificacao

Crie tambem 'testa_validacoes.py' que importa o modulo e testa cada funcao com pelo menos 3 entradas diferentes (validas e invalidas), exibindo os resultados no terminal.

Dica: use try/except nas funcoes para tratar entradas inesperadas.

AULA 7

Testes Unitarios com pytest

7.1 O que e pytest?

pytest e o framework de testes mais popular do Python. Ele permite escrever testes de forma simples, executar varios testes de uma vez e gerar relatorios detalhados.

Caracteristica	Detalhe
Simplicidade	Testes sao funcoes simples comecando com 'test_'
Auto-descoberta	pytest encontra e executa todos os testes automaticamente
Mensagens de erro claras	Mostra exatamente o que falhou e por que
Fixtures	Mecanismo para preparar e limpar dados de teste
Plugins	Extensivel com pytest-html, pytest-cov, pytest-playwright e muito mais

7.2 Instalando o pytest

```
1 # No terminal do VS Code (Ctrl + `) execute:
2 pip install pytest
3
4 # Verificar instalacao:
5 pytest --version
6
7 # Instalar plugins uteis:
8 pip install pytest-html      # gera relatorio HTML
9 pip install pytest-cov      # mede cobertura de codigo
```

7.3 Escrevendo Seus Primeiros Testes

Primeiro, vamos criar o codigo que queremos testar. Salve como 'calculadora.py':

```
1 # calculadora.py - codigo a ser testado
2 def somar(a, b):
3     """Retorna a soma de dois numeros."""
4     return a + b
5
6 def subtrair(a, b):
7     """Retorna a subtracao de a por b."""
8     return a - b
9
10 def multiplicar(a, b):
11     """Retorna o produto de dois numeros."""
12     return a * b
13
```

```
14 def dividir(a, b):
15     """Retorna a divisao de a por b.
16     Raises:
17         ValueError: se b for zero
18     """
19     if b == 0:
20         raise ValueError("Divisao por zero nao e permitida")
21     return a / b
```

Agora crie os testes. Salve como 'test_calculadora.py' (IMPORTANTE: o arquivo DEVE começar com 'test_'):

```
1 # test_calculadora.py - arquivo de testes
2 import pytest
3 from calculadora import somar, subtrair, multiplicar, dividir
4
5 # === TESTES DE SOMA ===
6 def test_somar_numeros_positivos():
7     resultado = somar(3, 5)
8     assert resultado == 8
9
10 def test_somar_numero_negativo():
11     resultado = somar(-2, 5)
12     assert resultado == 3
13
14 def test_somar_zeros():
15     assert somar(0, 0) == 0
16
17 # === TESTES DE DIVISAO ===
18 def test_dividir_resultado_correto():
19     assert dividir(10, 2) == 5.0
20
21 def test_dividir_por_zero_levanta_erro():
22     with pytest.raises(ValueError) as excinfo:
23         dividir(10, 0)
24     assert "zero" in str(excinfo.value)
25
26 # === TESTE COM MULTIPLOS VALORES (parametrize) ===
27 @pytest.mark.parametrize("a, b, esperado", [
28     (1, 1, 2),
29     (10, 20, 30),
30     (-5, 5, 0),
31     (0, 100, 100),
32 ])
33 def test_somar_varios_valores(a, b, esperado):
34     assert somar(a, b) == esperado
```

7.4 Executando os Testes

```
1 # Executar todos os testes na pasta atual
2 pytest
3
4 # Executar com detalhes (verbose)
5 pytest -v
6
7 # Executar um arquivo especifico
8 pytest test_calculadora.py
9
10 # Executar um teste especifico
11 pytest test_calculadora.py::test_somar_numeros_positivos
12
13 # Gerar relatorio HTML
14 pytest --html=relatorio.html --self-contained-html
15
16 # Ver cobertura de codigo
17 pytest --cov=calculadora --cov-report=term-missing
```

7.5 Entendendo o Resultado do pytest

```
1 # Exemplo de saida do pytest -v:
2 # ===== test session starts =====
3 # collected 8 items
4 #
5 # test_calculadora.py::test_somar_numeros_positivos PASSED [ 12%]
6 # test_calculadora.py::test_somar_numero_negativo PASSED [ 25%]
7 # test_calculadora.py::test_somar_zeros PASSED [ 37%]
8 # test_calculadora.py::test_dividir_resultado_correto PASSED [ 50%]
9 # test_calculadora.py::test_dividir_por_zero_levanta_erro PASSED [ 62%]
10 # test_calculadora.py::test_somar_varios_valores[1-1-2] PASSED [ 75%]
11 # test_calculadora.py::test_somar_varios_valores[10-20-30] PASSED [ 87%]
12 # test_calculadora.py::test_somar_varios_valores[-5-5-0] PASSED [100%]
13 #
14 # ===== 8 passed in 0.12s =====
```



Dica

PASSOU = ponto verde . | FALHOU = F vermelho | ERRO = E amarelo

Quando um teste FALHA, o pytest mostra exatamente qual valor era esperado e qual foi obtido. Isso facilita muito a identificacao do bug.

7.6 Fixtures: Preparando o Ambiente de Teste

```
1 # Fixtures evitam repeticao de codigo de preparacao
2 import pytest
3 from validacoes import validar_email
4
5 @pytest.fixture
6 def emails_validos():
```

```
7      """Retorna uma lista de emails validos para teste."""
8      return [
9          "usuario@email.com",
10         "nome.sobrenome@empresa.com.br",
11         "contato@senai.org",
12     ]
13
14     @pytest.fixture
15     def emails_invalidos():
16         return ["semArroba", "@semdominio.com", "", "sem@pontonodominio"]
17
18     def test_emails_validos_passam(emails_validos):
19         for email in emails_validos:
20             assert validar_email(email) == True, f"Deveria ser valido: {email}"
21
22     def test_emails_invalidos_falham(emails_invalidos):
23         for email in emails_invalidos:
24             assert validar_email(email) == False, f"Deveria ser invalido: {email}"
```

7.7 Atividade Pratica - Aula 7

* Atividade 7.1 - TDD: Test Driven Development

Vamos praticar TDD (escrever o teste ANTES do código):

Crie testes para as funções abaixo (sem implementá-las ainda):

1. `calcular_media(notas: list) -> float`
2. `verificar_aprovacao(media: float, frequencia: float) -> dict`
(aprovado se `media >= 7` E `frequencia >= 75%`)
3. `formatar_cpf(cpf: str) -> str`
(transforma `'12345678900'` em `'123.456.789-00'`)

Etapa 1: Execute os testes - todos devem FALHAR (funções não existem ainda)

Etapa 2: Implemente as funções até todos os testes PASSAREM

Etapa 3: Gere o relatório HTML com `pytest --html=relatorio_aula7.html`

AULA 8

Testes de Interface Web com Playwright

8.1 O que é Playwright?

Playwright é uma ferramenta da Microsoft para automação de navegadores. Com ela, você escreve código Python que controla o navegador como se fosse um usuário real: clica em botões, preenche formulários, navega entre páginas e verifica resultados.

Característica	Playwright vs alternativas
Velocidade	Playwright é mais rápido que Selenium — usa protocolo WebSocket diretamente
Suporte a navegadores	Chrome, Firefox e Safari em uma única API
Auto-wait	Espera automaticamente elementos ficarem disponíveis — sem <code>sleep()</code> manual
Screenshots e video	Captura evidências automaticamente em caso de falha
Modo headless	Executa sem abrir janela visível — ideal para CI/CD
Instalação simples	Um comando instala tudo incluindo os navegadores

8.2 Instalando o Playwright

```
1 # Instalar o Playwright e o plugin para pytest
2 pip install playwright pytest-playwright
3
4 # Instalar os navegadores (Chrome, Firefox, Safari)
5 playwright install
6
7 # Verificar instalação
8 playwright --version
```

8.3 Seu Primeiro Teste com Playwright

Vamos criar um teste que acessa o site do SENAI e verifica se o título da página está correto:

```
1 # test_web_basico.py
2 from playwright.sync_api import Page, expect
3
4 def test_titulo_pagina_senai(page: Page):
5     """Verifica se a página do SENAI carrega corretamente."""
6     page.goto("https://www.senai.br")
7     expect(page).to_have_title("SENAI")
8
9 def test_campo_busca_existe(page: Page):
10     """Verifica se o campo de busca está presente."""
```

```
11 page.goto("https://www.senai.br")
12 campo_busca = page.locator("input[type='search']")
13 expect(campo_busca).to_be_visible()
```

```
1 # Executar testes Playwright (modo headless padrao):
2 pytest test_web_basico.py
3
4 # Ver o navegador abrindo (modo headed):
5 pytest test_web_basico.py --headed
6
7 # Executar em Firefox:
8 pytest test_web_basico.py --browser firefox
```

8.4 Interagindo com Elementos da Pagina

```
1 # test_formulario.py
2 from playwright.sync_api import Page, expect
3
4 def test_login_credenciais_validas(page: Page):
5     """Testa login com email e senha corretos."""
6     # Navegar para a pagina de login
7     page.goto("http://localhost:3000/login")
8
9     # Encontrar e preencher campos
10    page.fill("input[type='email']", "admin@teste.com")
11    page.fill("input[type='password']", "Senha@123")
12
13    # Clicar no botao de login
14    page.click("button[type='submit']")
15
16    # Verificar redirecionamento
17    expect(page).to_have_url("http://localhost:3000/dashboard")
18
19    # Verificar elemento que so aparece apos login
20    expect(page.locator(".user-greeting")).to_be_visible()
21
22 def test_login_senha_errada(page: Page):
23     """Testa mensagem de erro com senha incorreta."""
24    page.goto("http://localhost:3000/login")
25    page.fill("input[type='email']", "admin@teste.com")
26    page.fill("input[type='password']", "senhaerrada")
27    page.click("button[type='submit']")
28
29    # Verificar mensagem de erro
30    mensagem_erro = page.locator(".error-message")
31    expect(mensagem_erro).to_be_visible()
32    expect(mensagem_erro).to_contain_text("Email ou senha incorretos")
33
```

```

34 def test_campo_email_vazio(page: Page):
35     page.goto("http://localhost:3000/login")
36     page.fill("input[type='password']", "Senha@123")
37     page.click("button[type='submit']")
38     expect(page.locator(".email-error")).to_contain_text("obrigatorio")

```

8.5 Capturando Evidencias Automaticamente

```

1  # confest.py - configuracao global de testes Playwright
2  import pytest
3  from playwright.sync_api import Page
4  import datetime
5
6  @pytest.fixture(autouse=True)
7  def screenshot_em_falha(page: Page, request):
8      """Tira screenshot automatico quando um teste falha."""
9      yield # executa o teste
10     if request.node.rep_call.failed:
11         timestamp = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
12         nome_teste = request.node.name
13         page.screenshot(path=f'evidencias/{nome_teste}_{timestamp}.png')
14         print(f'Screenshot salvo: evidencias/{nome_teste}_{timestamp}.png')
15
16 # Para habilitar videos de falha, use no pytest.ini:
17 # [pytest]
18 # addopts = --video=on --output=evidencias

```

8.6 Localizando Elementos (Seletores)

Seletor Playwright	Quando usar / Exemplo
<code>page.get_by_role()</code>	Busca por funcao ARIA. Ex: <code>get_by_role('button', name='Entrar')</code>
<code>page.get_by_text()</code>	Busca por texto visivel. Ex: <code>get_by_text('Bem-vindo')</code>
<code>page.get_by_label()</code>	Busca input pelo label associado. Ex: <code>get_by_label('Email')</code>
<code>page.get_by_placeholder()</code>	Busca pelo placeholder. Ex: <code>get_by_placeholder('Digite seu email')</code>
<code>page.locator('css')</code>	Seletor CSS. Ex: <code>locator('.menu-item')</code> ou <code>locator('#submit-btn')</code>
<code>page.locator('xpath')</code>	XPath. Ex: <code>locator('//button[@type="submit"]')</code>



Dica

Prefira `get_by_role()` e `get_by_label()` aos seletores CSS quando possivel. Eles sao mais resistentes a mudancas no HTML e tambem melhoram a acessibilidade do seu codigo de teste.

8.7 Atividade Pratica - Aula 8

*

Atividade 8.1 - Automatizando os Testes do Projeto Front End

Use o projeto de front end que voce criou na UC anterior.

Tarefa 1: Instale o Playwright e configure o conftest.py para captura automatica de screenshots

Tarefa 2: Converta os casos de teste manuais que voce criou na Aula 3 para testes automatizados com Playwright. Minimo de 6 testes.

Tarefa 3: Execute a suite completa com:

```
pytest --headed -v --html=relatorio_playwright.html --self-contained-html
```

Tarefa 4: Intencionalmente quebre um componente HTML (ex: remova o atributo 'type' de um input) e execute os testes novamente. Documente o que o Playwright detectou.

Entrega: Link GitHub com o codigo dos testes + arquivo relatorio_playwright.html

AULA 9

Relatorios de Teste Automatizados

9.1 A Importancia dos Relatorios de Teste

Um teste que passa silenciosamente e um teste que nao deixa rastro. Em projetos profissionais, TODA execucao de teste precisa gerar evidencias documentadas. Relatorios servem para:

- Comprovar que os testes foram executados (para o cliente e para a gestao)
- Facilitar a investigacao de falhas historicas
- Medir a evolucao da qualidade ao longo do projeto
- Alimentar metricas de qualidade (taxa de aprovacao, cobertura, bugs por modulo)

9.2 Relatorio HTML com pytest-html

```
1 # Gerando relatorio HTML completo
2 pytest -v --html=relatorio/relatorio_$(date +%Y%m%d).html --self-contained-html
3
4 # No Windows (PowerShell):
5 pytest -v --html=relatorio/relatorio.html --self-contained-html
```

Para personalizar o relatorio, adicione um arquivo conftest.py com metadados do projeto:

```
1 # conftest.py
2 import pytest
3 import datetime
4
5 def pytest_configure(config):
6     config._metadata = {
7         'Projeto': 'Sistema de Login - SENAI',
8         'Ambiente': 'Windows 11 / Chrome 125',
9         'Executado por': 'Equipe QA',
10        'Data': datetime.datetime.now().strftime('%d/%m/%Y %H:%M'),
11        'Versao': '1.0.0',
12    }
```

9.3 Cobertura deCodigo com pytest-cov

Cobertura de codigo mede qual percentual do seu codigo-fonte e executado pelos testes:

```
1 # Instalar pytest-cov
2 pip install pytest-cov
3
4 # Executar com relatorio de cobertura no terminal
5 pytest --cov=. --cov-report=term-missing
6
7 # Gerar relatorio HTML de cobertura
8 pytest --cov=. --cov-report=html
9 # Abre o arquivo htmlcov/index.html no navegador
```

```
10
11 # Combinar cobertura + relatorio HTML:
12 pytest -v --cov=. --cov-report=html --html=relatorio.html --self-contained-html
```

**Dica**

Meta de cobertura recomendada: 80%+ para código crítico de negócio. 100% é raramente necessário e pode indicar testes superficiais. O mais importante é que CENARIOS CRITICOS estejam cobertos.

9.4 Gerando Relatório de Teste em PDF

Para entregas formais, podemos gerar um relatório em formato de documento. Veja como criar um relatório personalizado em Python:

```
1 # gerador_relatorio.py
2 import json
3 import datetime
4 import subprocess
5
6 def executar_testes_e_salvar():
7     """Executa pytest e salva resultado em JSON."""
8     resultado = subprocess.run(
9         ['pytest', '-v', '--tb=short', '--json-report', '--json-report-file=resultado.json'],
10        capture_output=True, text=True
11    )
12    return resultado.returncode == 0
13
14 def gerar_sumario(arquivo_json):
15     """Le o JSON do pytest e gera sumario."""
16     with open(arquivo_json) as f:
17         dados = json.load(f)
18
19     total = dados['summary']['total']
20     passou = dados['summary'].get('passed', 0)
21     falhou = dados['summary'].get('failed', 0)
22
23     print(f"Total: {total} | Passou: {passou} | Falhou: {falhou}")
24     print(f"Taxa de sucesso: {(passou/total*100):.1f}%")
```

9.5 Metricas de Qualidade

Metrica	Como calcular / Interpretacao
Taxa de Aprovacao	Testes passados / Total de testes x 100. Meta: >95% em producao.
Densidade de Defeitos	Numero de bugs / Kloc (mil linhas de codigo). Indica qualidade do codigo.
Cobertura de Requisitos	Requisitos cobertos por testes / Total de requisitos x 100.
Tempo Medio de Deteccao	Tempo medio entre introducao e descoberta de um bug.

Taxa de Reincidencia

Bugs reabertos / Total de bugs fechados. Indica qualidade das correcoes.

9.6 Atividade Pratica - Aula 9

*

Atividade 9.1 - Relatorio Profissional de Teste

Execute a suite completa de testes do seu projeto e gere:

1. Relatorio HTML com pytest-html (personalizado com metadados do projeto)
2. Relatorio de cobertura de codigo (htmlcov/)
3. Um documento Word com o Relatorio Final de Teste contendo:
 - Identificacao do projeto
 - Resumo executivo (quantos testes, taxa de aprovacao)
 - Lista de bugs encontrados (do GitHub Issues)
 - Metricas calculadas
 - Conclusao e recomendacoes

AULA 10

Integracao Continua e Boas Praticas

10.1 O que e CI/CD?

#

Definicao

CI (Continuous Integration): pratica de integrar o codigo de todos os membros da equipe frequentemente (pelo menos uma vez por dia), com execucao automatica de testes a cada integracao.

CD (Continuous Delivery/Deployment): extensao do CI que automatiza tambem a entrega do software para os ambientes de homologacao ou producao.

Em termos praticos: toda vez que alguem faz um 'git push', os testes sao executados automaticamente. Se algum teste falhar, a equipe e notificada antes que o problema chegue a producao.

10.2 GitHub Actions: CI gratuito para estudantes

O GitHub Actions permite criar pipelines de CI/CD direto no GitHub, gratuitamente para repositorios publicos. Vamos configurar um pipeline que executa os testes Python automaticamente a cada push:

```
1 # Crie o arquivo: .github/workflows/testes.yml
2 name: Suite de Testes
3
4 on:
5   push:
6     branches: [ main, develop ]
7   pull_request:
8     branches: [ main ]
9
10 jobs:
11   testes:
12     runs-on: ubuntu-latest
13
14     steps:
15       - uses: actions/checkout@v4
16
17       - name: Configurar Python 3.12
18         uses: actions/setup-python@v5
19         with:
20           python-version: '3.12'
21
22       - name: Instalar dependencias
23         run: |
24           pip install pytest pytest-html pytest-cov
25           pip install playwright
```

```

26 playwright install --with-deps chromium
27
28 - name: Executar testes unitarios
29   run: pytest tests/ -v --html=relatorio.html --self-contained-html
30
31 - name: Publicar relatorio como artifact
32   uses: actions/upload-artifact@v4
33   if: always()
34   with:
35     name: relatorio-testes
36     path: relatorio.html

```

10.3 Boas Praticas de Testes

Princípio FIRST para testes de qualidade:

Princípio	Significado e Como Aplicar
F - Fast (Rápido)	Testes devem executar rapidamente. Evite acessos a banco real em testes unitarios — use mocks.
I - Isolated (Isolado)	Cada teste deve ser independente. A ordem de execucao nao deve importar.
R - Repeatable (Repetível)	O mesmo teste deve sempre ter o mesmo resultado, em qualquer maquina.
S - Self-validating	O proprio teste deve verificar se passou ou falhou — sem intervencao humana.
T - Timely (Oportuno)	Escreva testes junto com o codigo, nao depois. Melhor ainda: antes (TDD).

Nomeando Testes de Forma Clara:

```

1  # MAU exemplo: nao diz o que testa nem o que espera
2  def test_login():
3      ...
4
5  # BOM exemplo: DADO_QUANDO_ENTAO (Given_When_Then)
6  def test_dado_email_valido_quando_login_entao_redireciona_dashboard():
7      ...
8
9  # Ou o padrao mais curto igualmente legivel:
10 def test_login_com_credenciais_validas_redireciona_para_dashboard():
11     ...
12
13 def test_login_com_senha_errada_exibe_mensagem_de_erro():
14     ...

```

10.4 Organizacao de Projeto de Testes

```

1  meu-projeto/

```

```
2 | └─ src/          # codigo-fonte da aplicacao
3 |   └─ validacoes.py
4 |     └─ calculadora.py
5 | └─ tests/        # todos os testes aqui
6 |   └─ unit/       # testes unitarios
7 |     └─ test_validacoes.py
8 |       └─ test_calculadora.py
9 |   └─ integration/ # testes de integracao
10 |      └─ test_api.py
11 |   └─ e2e/        # testes de interface (Playwright)
12 |      └─ test_login_page.py
13 | └─ evidencias/   # screenshots e videos de teste
14 | └─ relatorios/   # relatorios HTML gerados
15 | └─ conftest.py   # configuracoes globais do pytest
16 | └─ pytest.ini    # configuracao do pytest
17 | └─ requirements.txt # dependencias do projeto
18 | └─ .github/workflows/ # pipelines de CI
19 |    └─ testes.yml
```

10.5 Atividade Pratica - Aula 10

* Atividade 10.1 - Configurando CI com GitHub Actions

Tarefa 1: Reorganize seu projeto seguindo a estrutura de pastas apresentada na secao 10.4

Tarefa 2: Crie o arquivo .github/workflows/testes.yml com o pipeline de CI

Tarefa 3: Faca commit e push para o GitHub e aguarde a execucao automatica

Tarefa 4: Verifique na aba 'Actions' do GitHub se o pipeline passou

Tarefa 5: Intencionalmente introduza um bug no codigo, faca push e documente o que o pipeline detectou

Entrega: Link do repositorio GitHub com o Actions configurado e funcionando + print da aba Actions

AULA 11

Projeto Integrador Final

11.1 Descrição do Projeto

Chegou a hora de colocar TUDO em prática! O Projeto Integrador Final é a atividade de fechamento desta UC. Você irá aplicar todos os conhecimentos adquiridos para criar uma suite completa de testes para um projeto web real.

#

Definição

O Projeto Integrador é avaliativo e representa 40% da nota final da UC. Pode ser realizado em duplas ou individualmente.

11.2 Requisitos do Projeto

Você deverá criar uma suite completa de testes para o projeto web que desenvolveu na UC de Codificação para Front End (ou um projeto indicado pelo professor). O projeto deve conter:

Documentação (30 pontos)

- Plano de Teste completo (seguindo a estrutura ISO 29119-3)
- Mínimo 10 casos de teste documentados (usando o modelo padrão)
- Mínimo 3 bug reports no GitHub Issues (com severidade e prioridade)
- Relatório Final de Teste em Word

Testes Automatizados (40 pontos)

- Mínimo 8 testes unitários com pytest (cobrindo funcionalidades Python)
- Mínimo 6 testes de interface com Playwright (cobrindo fluxos críticos)
- Cobertura de código $\geq 70\%$ (medida com pytest-cov)
- Uso correto de fixtures em pelo menos 2 funções

Qualidade e Organização (20 pontos)

- Estrutura de pastas organizada (src/, tests/unit/, tests/e2e/)
- Nomes descritivos para todos os testes
- Conftest.py com captura automática de screenshots em falhas
- Arquivo requirements.txt com todas as dependências

Pipeline CI/CD (10 pontos)

- GitHub Actions configurado e funcionando
- Pipeline executa os testes a cada push automaticamente
- Relatório HTML publicado como artifact no GitHub Actions

11.3 Critérios de Avaliação

Criterio	Peso
Documentacao (Plano + Casos + Bug Reports + Relatorio)	30%
Testes Automatizados (pytest + Playwright)	40%
Qualidade e Organizacao doCodigo	20%
Pipeline CI/CD (GitHub Actions)	10%

11.4 Cronograma Sugerido

Tempo	Atividade
Primeiras 2 horas	Plano de Teste + Casos de Teste documentados
2 horas seguintes	Implementacao dos testes unitarios (pytest)
2 horas seguintes	Implementacao dos testes Playwright
1 hora	Configuracao do confest.py e geracao de relatorios
1 hora final	GitHub Actions + revisao geral + apresentacao



Dica

Dica de ouro: comece sempre pela documentacao. Ao escrever os casos de teste ANTES de codificar os testes automatizados, voce tem mais clareza sobre o que precisa testar e evita retrabalho.

11.5 Checklist de Entrega

#

Checklist Final do Projeto Integrador

- ☐ Repositorio GitHub criado e publico
- ☐ Plano de Teste no repositorio (PDF ou Word)
- ☐ Casos de teste documentados (planilha ou Word)
- ☐ Minimo 3 bug reports como GitHub Issues
- ☐ Relatorio Final de Teste (Word)
- ☐ tests/unit/ com minimo 8 testes pytest
- ☐ tests/e2e/ com minimo 6 testes Playwright
- ☐ confest.py com screenshots automaticos
- ☐ pytest.ini configurado
- ☐ requirements.txt completo
- ☐ Cobertura >= 70% (print do relatorio)
- ☐ .github/workflows/testes.yml configurado
- ☐ Actions executando sem erro (print da aba Actions)
- ☐ Link do repositorio entregue ao professor

Referencias e Materiais de Apoio

Documentacao Oficial

- Python.org - Documentacao oficial: docs.python.org/3
- pytest - Framework de testes: docs.pytest.org
- Playwright Python: playwright.dev/python
- GitHub Actions: docs.github.com/en/actions

Leituras Recomendadas

- MYERS, Glenford J. The Art of Software Testing. 3. ed. Wiley, 2011.
- ISTQB Syllabus 2023 - Disponivel em bstqb.org.br (versao em portugues)
- ISO/IEC 29119 - Norma Internacional de Teste de Software

Certificacoes para Considerar no Futuro

Certificacao	Informacoes
CTFL - ISTQB	Certified Tester Foundation Level. A mais reconhecida no Brasil para QA.
CTAL-TM	Advanced Level Test Manager. Nivel avancado do ISTQB.
AWS Certified Developer	Relevante para testes em ambientes cloud.
GitHub Actions Certification	Certificacao oficial do GitHub para CI/CD.

Ferramentas Gratuitas para Continuar Praticando

Ferramenta	Link / Uso
The-Internet (Heroku)	the-internet.herokuapp.com - site de pratica para automacao
Automation Exercise	automationexercise.com - e-commerce para testes de pratica
JSONPlaceholder	jsonplaceholder.typicode.com - API fake para testes de integracao
Playwright Demo	demo.playwright.dev/todomvc - app de demonstracao do Playwright

Glossario Rapido

Termo	Definicao
Assert	Instrucao que verifica se uma condicao e verdadeira. Falha se nao for.
CI/CD	Integracao e entrega contínuas. Automacao de build, teste e deploy.
Coverage	Cobertura de codigo: percentual do codigo executado pelos testes.

Fixture	Funcao pytest que prepara dados ou estado para um ou mais testes.
Headless	Modo de execucao de navegador sem interface grafica visivel.
Mock	Objeto falso que simula o comportamento de um componente real.
QA	Quality Assurance. Area/profissional responsavel pela qualidade do software.
Regression	Teste que verifica se mudancas novas nao quebraram funcionalidades antigas.
Selector	Expressao usada para identificar elementos em uma pagina HTML.
TDD	Test Driven Development. Escrever o teste antes do codigo de producao.